

Лабораторная работа №2

Дисциплина: Математические основы защиты информации и информационной безопасности

Лабузов Александр Сергеевич

Содержание

Цель работы

Познакомиться с реализацией шифров перестановки: маршрутный, решётка, Вижинера.

Задание

1. Реализовать маршрутный шифр.
2. Реализовать шифрование решёткой.
3. Реализовать шифр Вижинера

Выполнение лабораторной работы

- 1) Сначала я написал функцию выбора алфавита.

```
# Алфавиты
ALPHABET_EN = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
ALPHABET_RU = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'

# Функция выбора алфавита:
def get_alphabet(language):
    if language.lower() == 'ru':
        return ALPHABET_RU
    elif language.lower() == 'en':
        return ALPHABET_EN
    else:
        raise ValueError("Недопустимый язык, Выберите ru или en")
```

Выбор алфавита

- 2) Далее реализован шифр Вижинера.

```

# Генерация ключа для шифра Вижинера
def generate_vigenere_key(text, key, alphabet):
    key = list(key)
    if len(key) == len(text):
        return key
    else:
        for i in range(len(text)-len(key)):
            key.append(key[i%len(key)])
        return "".join(key)

# Шифр Вижинера
def encrypt_vigenere(text, key, alphabet):
    key = generate_vigenere_key(text, key, alphabet)
    encrypted_text = ''
    for i in range(len(text)):
        text_index = alphabet.find(text[i])
        key_index = alphabet.find(key[i])
        if text_index == -1:
            encrypted_text += text[i] # Пропуск отсутствующих символов
        else:
            encrypted_text += alphabet[(text_index + key_index) % len(alphabet)]
    return encrypted_text

def decrypt_vigenere(encrypted_text, key, alphabet):
    key = generate_vigenere_key(encrypted_text, key, alphabet)
    decrypted_text = ''
    for i in range(len(encrypted_text)):
        text_index = alphabet.find(encrypted_text[i])
        key_index = alphabet.find(key[i])
        if text_index == -1:
            decrypted_text += encrypted_text[i] # Пропускаем символы, которых нет в алфавите
        else:
            decrypted_text += alphabet[(text_index - key_index + len(alphabet)) % len(alphabet)]
    return decrypted_text

```

3) Выполнение шифрования маршрутным шифром

```

# Маршрутное шифрование
def encrypt_route_cipher(text, rows, cols):
    # Удаляем пробелы и формируем текст
    text = text.replace(" ", "").upper()
    padding_len = rows * cols - len(text)
    text += 'X' * padding_len

    # Создаем матрицу
    matrix = []
    index = 0
    for r in range(rows):
        row = []
        for c in range(cols):
            row.append(text[index])
            index += 1
        matrix.append(row)

    # Чтение по столбцам
    encrypted_text = ''
    for c in range(cols):
        for r in range(rows):
            encrypted_text += matrix[r][c]

    return encrypted_text

```

Шифрование маршрутным

4) Выполнение дешифровки маршрутным шифром

```
def decrypt_route_cipher(encrypted_text, rows, cols):
    # Создаем матрицу по столбцам
    matrix = [[''] * cols for _ in range(rows)]
    index = 0
    for c in range(cols):
        for r in range(rows):
            matrix[r][c] = encrypted_text[index]
            index += 1

    # Чтение по строкам
    decrypted_text = ""
    for r in range(rows):
        decrypted_text += ''.join(matrix[r])

    return decrypted_text
```

Дешифровка маршрутным

5) Выполнение шифрования решёткой:

```
# шифрование с помощью решетки
def create_grid(text, key):
    size = int(math.sqrt(len(key)))
    text_grid = [['' for _ in range(size)] for _ in range(size)]
    idx = 0
    for (i, j) in key:
        text_grid[i][j] = text[idx]
        idx += 1
    return text_grid

def encrypt_grid_cipher(text, key):
    size = int(math.sqrt(len(key)))
    if len(text) < size * size:
        text += 'X' * (size * size - len(text))

    text_grid = create_grid(text, key)
    encrypted_text = ""
    for row in text_grid:
        encrypted_text += ''.join(row)
    return encrypted_text

def decrypt_grid_cipher(encrypted_text, key):
    size = int(math.sqrt(len(key)))
    grid = [[''] * size for _ in range(size)]
    idx = 0
    for i, j in key:
        grid[i][j] = encrypted_text[idx]
        idx += 1
    decrypted_text = ""
```

Шифр решётки

6) Выполнение основной части программы (взаимодействия с пользователем)

```

# Основная программа
def main():
    print("Выберите язык (ru - русский, en - английский):")
    language = input().strip().lower()

    # Получаем алфавит для выбранного языка
    try:
        alphabet = get_alphabet(language)
    except ValueError as e:
        print(e)
        return

    print("Выберите метод шифрования: 1) Вижинера 2) Маршрутное 3) Решётка")
    method = input().strip()

    if method == "1":
        # Шифр Вижинера
        print("Введите текст:")
        text = input().strip().upper()
        print("Введите ключ:")
        key = input().strip().upper()

        encrypted = encrypt_vigenere(text, key, alphabet)
        decrypted = decrypt_vigenere(encrypted, key, alphabet)

        print(f"Зашифрованный текст: {encrypted}")
        print(f"Расшифрованный текст: {decrypted}")

```

Взаимодействие с пользователем

7) Проверка работоспособности шифра Вижинера

```

Выберите язык (ru - русский, en - английский):
ru
Выберите метод шифрования: 1) Вижинера 2) Маршрутное 3) Решётка
1
Введите текст:
Лекарственныйсон
Введите ключ:
5
Зашифрованный текст: K
Расшифрованный текст: Л

```

Проверка метода шифра Вижинера

8) Проверка работоспособности маршрутного шифра

![Проверка маршрутного шифра]](image1/image9.png){ width=70% }

Полный код программы в <https://colab.google>:

[https://colab.research.google.com/drive/1mA9WtslytuM3D3ZfgU4SkrwKG8rGgdXR?usp=s](https://colab.research.google.com/drive/1mA9WtslytuM3D3ZfgU4SkrwKG8rGgdXR?usp=ssharing)
haring

Выводы

На практике изучил работу познакомиться с шифрами перестановки: маршрутный, решётка, Вижинера.